Adatbázis és PL/SQL Fogalomtár – Próba ZH (ZH2)

Ez a dokumentum a `proba_zh.pdf` feladatsorában előforduló összes adatbázis- és PL/SQL fogalmat, azok pontos definícióját és a gyakorlatban leggyakrabban használt mintakódjait foglalja össze Oracle XE / PL/SQL Developer környezetre adaptálva.

---

1. Adatbázis jogosultságok (Database Privileges / `GRANT`)

- **Definíció:** Az adatbázis-kezelőben a felhasználók (sémák) számára engedélyezett műveletek köre (pl. DDL utasítások futtatása objektumok létrehozására). Típusok (`OBJECT TYPE`) létrehozásához külön rendszerszintű jogosultság (`CREATE TYPE`) szükséges, amelyet emelt szintű (pl. SYSDBA) felhasználó adhat ki.
- **Leggyakoribb példa (SYS felhasználóval futtatva):** `sql GRANT CREATE TYPE TO termek_nyilvantarto; GRANT CREATE PROCEDURE TO termek_nyilvantarto; GRANT CREATE TABLE TO termek_nyilvantarto; GRANT CREATE SESSION TO termek_nyilvantarto;`

---

2. Objektum típus (Object Type / `CREATE TYPE ... AS OBJECT`)

- **Definíció:** Felhasználó által definiált összetett adattípus (Composite Data Type / UDT), amely több logikailag összetartozó attribútumot (és opcionálisan metódusokat) fog össze egyetlen egységbe. SQL táblák oszloptípusaként, valamint PL/SQL változóként és visszatérési értékként is használható.
- **Leggyakoribb példa:** `sql CREATE OR REPLACE TYPE ty_termek_type AS OBJECT ( termek_id NUMBER, megnevezes VARCHAR2(250) ); /`

---

3. Konstruktor függvény (Constructor Function)

- **Definíció:** Az objektum típusok példányosítására és inicializálására szolgáló speciális függvény. Alapértelmezés szerint az Oracle minden attribútumot azonos sorrendben váró konstruktort hoz létre, de definiálható egyedi (pl. paraméter nélküli vagy részleges) konstruktor is `CONSTRUCTOR FUNCTION ... RETURN SELF AS RESULT` szignatúrával.
- **Leggyakoribb példa:** ```sql -- Típus specifikációban: CREATE OR REPLACE TYPE ty_termek_type AS OBJECT (termek_id NUMBER, megnevezes VARCHAR2(250), CONSTRUCTOR FUNCTION ty_termek_type RETURN SELF AS RESULT); /

-- Típus törzsben (TYPE BODY): CREATE OR REPLACE TYPE BODY ty_termek_type AS CONSTRUCTOR FUNCTION ty_termek_type RETURN SELF AS RESULT IS BEGIN SELF.termek_id := NULL; SELF.megnevezes := NULL; RETURN; END; END; / ```

---

4. Tárolt függvény (Stored Function / `CREATE FUNCTION`)

- **Definíció:** Névvel ellátott, előrefordított PL/SQL programegység, amely bemeneti paramétereket fogad, műveleteket végez, és **mindig pontosan egy visszatérési értékkel** (`RETURN`) tér vissza. SQL lekérdezésekből (`SELECT`) is meghívható, amennyiben nem hajt végre tiltott adatbázis-módosítást (DML).
- **Leggyakoribb példa:** ```sql CREATE OR REPLACE FUNCTION get_termek_adatok(p_id IN NUMBER) RETURN ty_termek_type IS v_id NUMBER; v_nev VARCHAR2(250); BEGIN SELECT id, megnevezes INTO v_id, v_nev FROM termek WHERE id = p_id;

  RETURN ty_termek_type(v_id, v_nev); EXCEPTION WHEN NO_DATA_FOUND THEN RETURN ty_termek_type(); -- paraméter nélküli konstruktor WHEN TOO_MANY_ROWS THEN RETURN NULL; END get_termek_adatok; / ```

---

5. Kivételkezelés (Exception Handling)

- **Definíció:** Futásidejű hibák (runtime errors) elkapására és kezelésére szolgáló vezérlési szerkezet (`EXCEPTION` blokk). Megakadályozza a program váratlan leállását, és lehetővé teszi egyedi hibakódok, alapértelmezett értékek vagy naplóüzenetek visszaadását.
- **A feladatsor kulcskivételei:**
- `NO_DATA_FOUND`: A `SELECT ... INTO` lekérdezés egyetlen rekordot sem talált.
- `TOO_MANY_ROWS`: A `SELECT ... INTO` egynél több rekordot talált.
- **Leggyakoribb példa:** ```sql BEGIN SELECT egysegar, keszlet INTO v_egysegar, v_keszlet FROM bolt_termek WHERE id = p_bt_id;

  RETURN v_egysegar * v_keszlet; EXCEPTION WHEN NO_DATA_FOUND THEN RETURN -1; WHEN TOO_MANY_ROWS THEN RETURN -2; END; ```

---

6. Csoportosító szöveg-összefűzés (`LISTAGG`)

- **Definíció:** Beépített Oracle SQL aggregáló függvény, amely több sor adott szöveges oszlopának értékeit összefűzi egyetlen sztringgé egy megadott határolójel mentén, meghatározott rendezés (`WITHIN GROUP (ORDER BY ...)`) szerint.
- **Leggyakoribb példa:**
  `sql SELECT LISTAGG(t.megnevezes, ',') WITHIN GROUP (ORDER BY t.megnevezes) INTO v_result FROM bolt_termek bt JOIN termek t ON t.id = bt.termek_id WHERE bt.bolt_id = p_bolt_id;`

---

7. Csomag (Package / Fejléc és Törzs)

- **Definíció:** Logikailag összefüggő eljárások, függvények, típusok, kurzorok és változók egységbe zárása (moduláris programozás).
- **Fejléc (`PACKAGE`):** A publikus interfész; csak a kívülről hívható elemek deklarációját tartalmazza.
- **Törzs (`PACKAGE BODY`):** Az implementáció, valamint a privát segédfüggvények, eljárások és változók.

- **Leggyakoribb példa:** ```sql -- Fejléc: CREATE OR REPLACE PACKAGE pkg_raktar AS gv_alap_keszlet NUMBER;

  FUNCTION get_keszlet_ertek(p_bt_id IN NUMBER) RETURN NUMBER; PROCEDURE ar_kalkulacio(p_bt_id IN NUMBER); PROCEDURE print_termek_boltok(p_termek_id IN NUMBER); END pkg_raktar; /

-- Törzs: CREATE OR REPLACE PACKAGE BODY pkg_raktar AS FUNCTION get_keszlet_ertek(p_bt_id IN NUMBER) RETURN NUMBER IS v_keszlet NUMBER; v_ar NUMBER; BEGIN SELECT keszlet, egysegar INTO v_keszlet, v_ar FROM bolt_termek WHERE id = p_bt_id; RETURN v_keszlet * v_ar; EXCEPTION WHEN NO_DATA_FOUND THEN RETURN -1; WHEN TOO_MANY_ROWS THEN RETURN -2; END get_keszlet_ertek;

```
PROCEDURE ar_kalkulacio(p_bt_id IN NUMBER) IS
BEGIN
  NULL; -- implementáció
END ar_kalkulacio;

PROCEDURE print_termek_boltok(p_termek_id IN NUMBER) IS
BEGIN
  NULL; -- implementáció
END print_termek_boltok;
```

END pkg_raktar; / ```

---

8. Csomagszintű változó és Inicializációs blokk (Package Variable & Initialization)

- **Definíció:**
- **Csomagszintű változó:** A csomag fejlécében deklarált globális változó, amelynek értéke a munkamenet (session) ideje alatt mindvégig perzisztens marad.
- **Inicializációs blokk:** A csomagtörzs végén elhelyezett `BEGIN ... END;` blokk, amely pontosan egyszer hajtódik végre automatikusan, amikor a munkamenet legelőször hivatkozik a csomagra.
- **Leggyakoribb példa:** ```sql CREATE OR REPLACE PACKAGE BODY pkg_raktar AS -- eljárások és függvények...

-- Inicializációs blokk a törzs legvégén: BEGIN gv_alap_keszlet := 100; END pkg_raktar; / ```

---

9. Tárolt eljárás (Stored Procedure / `CREATE PROCEDURE`)

- **Definíció:** Önmagában lefordított és tárolt PL/SQL programegység, amely műveleteket hajt végre (pl. DML frissítést, naplózást, kiíratást). Nincs közvetlen `RETURN` értéke (bár `OUT` vagy `IN OUT` paraméterekkel adatot tud visszaadni).
- **Leggyakoribb példa:** ```sql CREATE OR REPLACE PROCEDURE ar_kalkulacio(p_bt_id IN NUMBER) IS v_keszlet NUMBER; BEGIN SELECT keszlet INTO v_keszlet FROM bolt_termek WHERE id = p_bt_id;

  IF v_keszlet < pkg_raktar.gv_alap_keszlet THEN UPDATE bolt_termek SET egysegar = egysegar * 1.1 WHERE id = p_bt_id; END IF; EXCEPTION WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Hiba: nincs adat a megadott azonositoval.'); WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('Hiba: tul sok talalt.'); END ar_kalkulacio; / ```

10. Kurzor (Cursor / Cursor FOR LOOP)

- **Definíció:** Lekérdezések több sorból álló eredményhalmazának soronkénti bejárására és feldolgozására szolgáló adatbázis mutató. A `FOR rec IN cur LOOP` forma automatikusan megnyitja a kurzort (`OPEN`), egyenként beolvassa a sorokat (`FETCH`), majd a ciklus végén le is zárja azt (`CLOSE`).
- **Leggyakoribb példa:** ```sql PROCEDURE print_termek_boltok(p_termek_id IN NUMBER) IS v_nev VARCHAR2(250);

  CURSOR cur_boltok IS SELECT b.nev FROM bolt_termek bt JOIN bolt b ON b.id = bt.bolt_id WHERE bt.termek_id = p_termek_id; BEGIN SELECT megnevezes INTO v_nev FROM termek WHERE id = p_termek_id; DBMS_OUTPUT.PUT_LINE('Termek neve: ' || v_nev);

  FOR rec IN cur_boltok LOOP DBMS_OUTPUT.PUT_LINE('Bolt: ' || rec.nev); END LOOP; EXCEPTION WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Hiba: nem talalhato termek.'); END print_termek_boltok; ```

11. Konzol kimenet (`DBMS_OUTPUT.PUT_LINE`)

- **Definíció:** Az Oracle beépített rendszercsomagja, amely a PL/SQL kód futása közben szöveges sorokat ír a standard kimeneti pufferbe. A fejlesztői eszközök (pl. PL/SQL Developer Output füle, SQL*Plus `SERVEROUTPUT ON` mellett) ezt jelenítik meg a felhasználónak.
- **Leggyakoribb példa:** `sql DBMS_OUTPUT.PUT_LINE('Termek neve: ' || v_nev); DBMS_OUTPUT.PUT_LINE('Hiba: nem talalhato termek.');`

12. Feltételes vezérlés (Conditional Logic / `IF-THEN-ELSIF-ELSE`)

- **Definíció:** Logikai feltételektől függő ágválasztás a PL/SQL logikában (pl. rekordok darabszámának vizsgálata 0, 1 és 1-nél több elem esetén).
- **Leggyakoribb példa:** `sql IF v_count = 0 THEN RETURN 'Nincs termek!'; ELSIF v_count = 1 THEN SELECT megnevezes INTO v_egy_nev FROM termek WHERE id = p_id; RETURN v_egy_nev; ELSE SELECT LISTAGG(megnevezes, ',') WITHIN GROUP (ORDER BY megnevezes) INTO v_lista FROM termek; RETURN v_lista; END IF;`

---

13. DML adatmódosítás PL/SQL-ben (`UPDATE`)

- **Definíció:** Táblában létező sorok adatainak módosítása tranzakció keretében. PL/SQL-ben közvetlenül használható SQL utasítás.
- **Leggyakoribb példa:** `sql UPDATE bolt_termek SET egysegar = egysegar * 1.1 WHERE id = p_bt_id;`

Fogalmak összefoglaló táblázata

Fogalom	Kulcsszó / Szintaxis	Fő funkció
Jogosultság	`GRANT CREATE TYPE TO ...`	Sémának engedélyezi objektumok létrehozását
Objektum típus	`CREATE TYPE ... AS OBJECT`	Összetett attribútum-struktúra definiálása
Konstruktor	`CONSTRUCTOR FUNCTION ... RETURN SELF AS RESULT`	Típus inicializálása (pl. üres példány)
Függvény	`CREATE FUNCTION ... RETURN ...`	Értéket számító és visszaadó alprogram
Kivételkezelés	`EXCEPTION WHEN ... THEN`	Futási hibák (`NO_DATA_FOUND`, stb.) elkapása
Összefűzés	`LISTAGG(...) WITHIN GROUP (...)`	Csoportosított értékek szöveges összefűzése
Csomag fejléc	`CREATE PACKAGE ... AS`	Publikus interfész definíciója
Csomag törzs	`CREATE PACKAGE BODY ... AS`	Implementáció és belső logika megvalósítása
Csomag változó	`gv_valtozo NUMBER;`	Session szinten megmaradó állapot
Csomag inicializáció	`BEGIN gv_val := 100; END pkg;`	Első hivatkozáskor egyszer lefutó kód
Eljárás	`CREATE PROCEDURE ...`	Tevékenységet, adatmódosítást végző alprogram

Fogalom	Kulcsszó / Szintaxis	Fő funkció
Kurzor	`CURSOR cur IS ... / FOR r IN cur LOOP`	Lekérdezések több sorának feldolgozása
Konzol kimenet	`DBMS_OUTPUT.PUT_LINE(...)`	Hiba- és tájékoztató üzenetek kiírása
Feltételes vezérlés	`IF ... ELSIF ... ELSE ... END IF;`	Logikai feltételek szerinti elágazás